



TRABALHO DE OFICINA DE INTEGRAÇÃO

Spy Car Controlled by Android Application

ODILON RAMOS DA SILVA NETO¹

VITHOR AUGUSTO MOHR²

YURI MATSUMOTO SANTOS³

PATO BRANCO - PR
JULHO/2025

¹ Graduando em Engenharia de Computação pela UTFPR-PB. email: odilonneto@alunos.utfpr.edu.br

² Graduando em Engenharia de Computação pela UTFPR-PB. email: vithor.2003@alunos.utfpr.edu.br

³ Graduando em Engenharia de Computação pela UTFPR-PB. email: yurimatsumoto@alunos.utfpr.edu.br



RESUMO

Este artigo detalha o desenvolvimento de um protótipo funcional de um veículo de espionagem (Spy Car) controlado remotamente por um aplicativo Android, com transmissão de vídeo em tempo real. O sistema integra um microcontrolador ESP32-CAM, que é responsável pela captura de imagens e controle dos servomotores, com a plataforma Firebase Realtime Database, que é responsável pelo intermédio da troca de comandos de baixa latência. Os comandos de movimentação do veículo e da câmera são enviados a partir de uma interface intuitiva no aplicativo, que inclui um joystick virtual, botões de controle e um botão de ligar o LED. O vídeo é transmitido do ESP32-CAM para um servidor web dedicado em Python, Flask com Socket.IO, que retransmite para um componente WebView no aplicativo Android. Essa abordagem otimiza o desempenho do microcontrolador, como também garante escalabilidade ao sistema permitindo a visualização do aplicativo por múltiplos clientes. Os resultados demonstram uma solução de baixo custo, robusta e eficaz para aplicações de monitoramento remoto e robótica móvel.

ABSTRACT

This paper details the development of a functional prototype of a spy vehicle (Spy Car) controlled remotely by an Android application, with real-time video transmission. The system integrates an ESP32-CAM microcontroller, which is responsible for capturing images and controlling the servomotors, with the Firebase Realtime Database platform, which is responsible for the exchange of low-latency commands. The vehicle and camera movement commands are sent from an intuitive interface in the application, which includes a virtual joystick, control buttons and a button to turn on the LED. The video is transmitted from the ESP32-CAM to a dedicated web server in Python, Flask with Socket.IO, which relays it to a WebView component in the Android application. This approach optimizes the performance of the microcontroller, as well as guarantees scalability to the system, allowing the application to be viewed by multiple clients. The results demonstrate a low-cost, robust and effective solution for remote monitoring and mobile robotics applications.



1. INTRODUÇÃO

A alta disseminação da internet das coisas e de sistemas embarcados tornaram os desenvolvimentos da solução de monitoramento e controle remoto por smartphone possíveis. Nesse sentido, o módulo ESP32CAM é distinto por atender a conectividade Wi-Fi, processamento e capacidade de câmera incorporada, o que oferece uma plataforma de custo reduzido para visão computacional e automação de projetos. Mas para explorar todo o seu potencial para a aplicação móvel, é necessário então desenvolver uma interface simples e ao mesmo tempo responsiva, capaz de converter comandos do usuário em comandos do ESP32CAM, além de apresentar um livestream com baixo tempo de latência para o usuário.

Assim, este trabalho proposto visa desenvolver um aplicativo Android que envia comandos via Firebase para controle de dois servomotores que controlarão a rotação da câmera de um módulo ESP32CAM e de dois motores DC de um veículo robótico, recebendo e exibindo o stream de vídeo em tempo real da câmera. A arquitetura geral consiste no funcionamento pelo Firebase Realtime Database como forma de comunicação entre app e ESP32CAM, proporcionando facilidade na implementação e sustentabilidade do sistema.

2. TRABALHOS RELACIONADOS

Diversos projetos e pesquisas na área de carrinhos eletrônicos com câmeras abordam a interface entre usuário e o robô. Entre os trabalhos relacionados, destacam-se:

Projeto de controle remoto por Wi-Fi utilizando ESP32-CAM

Projeto de controle remoto no qual os autores utilizam o módulo para capturar vídeo em tempo real e controlam a movimentação da câmera por meio de uma interface web. Este projeto demonstrou a viabilidade da transmissão de vídeo e controle simultâneo, porém apresenta limitações quanto à responsividade da interface e à latência na comunicação direta via rede local. Nosso projeto supera essa limitação ao utilizar um servidor web intermediário, permitindo acesso global ao feed de vídeo.



Sistema de vigilância robótico com controle via Bluetooth e câmera embarcada

Que utiliza um aplicativo Android para movimentar um robô e visualizar imagens capturadas por uma câmera IP. Embora o uso de Bluetooth reduza a complexidade da comunicação, limita-se ao alcance físico e não se beneficia das vantagens de uma comunicação em nuvem como o Firebase.

Plataformas baseadas em ESP32 com controle por Blynk

Que demonstram a facilidade de integração entre módulos ESP e aplicativos móveis, ainda que dependam de serviços de terceiros e apresentam menor flexibilidade no design da interface do usuário.

3. REFERENCIAL TEÓRICO

Internet das Coisas (IoT)

A Internet das Coisas (IoT) é definida como a conexão entre a parte digital de objetos físicos com a internet permitindo a coleta e o envio e a recepção de dados em tempo real. Aplicações como automação residencial, cidades inteligentes e sistemas de monitoramento dependem fortemente dessa integração entre hardware e software.

ESP32-CAM

O ESP32-CAM é um módulo que combina um microcontrolador com conectividade Wi-Fi/Bluetooth e uma interface para câmera. Além de baixo custo, oferece recursos e capacidade de processamento adequada para aplicações embarcadas com visão computacional simples. Sua arquitetura o torna ideal para projetos que exigem captura de imagem e transmissão remota, como vigilância e robótica.

Controle de robôs móveis

O controle de robôs móveis requer uma interface eficiente entre o usuário e os atuadores do robô. Portanto, soluções que integram transmissão de vídeo e controle simultâneo devem minimizar atrasos, especialmente em sistemas em tempo real.



Plataformas em nuvem e Firebase

O Firebase Realtime Database é uma plataforma de backend, desenvolvida pelo Google, que permite a sincronização em tempo real de dados entre dispositivos. E o uso em aplicações é vantajoso pela simplicidade na integração, atualização rápida dos dados e compatibilidade com diferentes linguagens e plataformas.

Desenvolvimento de aplicativos móveis

A criação de aplicativos Android para controle de sistemas embarcados exige a implementação de interfaces amigáveis e eficientes, capazes de traduzir ações do usuário em comandos compreendidos pelo hardware.

Streaming de Vídeo com Flask e WebSocket

O ESP32-CAM captura um frame em formato *JPEG* e o envia via uma requisição *HTTP POST* para um servidor web. Este servidor, desenvolvido em Python com o microframework Flask, recebe a imagem, a codifica e a envia para todos os clientes conectados através do protocolo WebSocket, utilizando a biblioteca Socket.IO.

4. METODOLOGIA

A metodologia deste projeto foi dividida em quatro etapas principais: a montagem do hardware do veículo, o desenvolvimento do firmware para o microcontrolador, a implementação do servidor de streaming e a criação do aplicativo de controle para Android.

Hardware

A base do protótipo consiste em um chassi de veículo robótico com dois motores DC para tração e dois servomotores para a movimentação da câmera nos eixos horizontal e vertical. O cérebro do sistema é o módulo ESP32-CAM, escolhido por sua capacidade de processamento, conectividade Wi-Fi integrada e interface para câmera. Os motores DC são controlados por uma ponte H L298N, que permite o controle de direção e velocidade, enquanto os servomotores são conectados diretamente às portas GPIO do ESP32.



Firmware (ESP32-CAM)

O firmware foi desenvolvido no Arduino IDE, utilizando bibliotecas para a câmera, conectividade Wi-Fi e comunicação com o Firebase. O microcontrolador conecta-se à rede Wi-Fi e estabelece um listener em tempo real no nó “/comandos” do Firebase Realtime Database. Uma função de callback “streamCallback” interpreta os comandos recebidos (ex: “forward”, “left”, “up”) e os traduz em sinais PWM para os motores e ângulos para os servos. Simultaneamente, o ESP32-CAM captura os frames de vídeo, no formato JPEG, e os envia via requisições HTTP POST para um servidor web dedicado.

Servidor de Streaming (Python e Flask)

Para gerenciar o fluxo de vídeo, foi implementado um servidor web utilizando o microframework Flask em Python. O servidor possui um endpoint “/upload” que recebe os dados de imagem do ESP32-CAM. Ao receber um frame, ele o codifica em Base64 e o retransmite imediatamente para todos os clientes conectados através do protocolo WebSocket, implementado com a biblioteca Flask-SocketIO. Essa abordagem desacoplada otimiza o desempenho, pois o microcontrolador apenas envia os dados, sem se preocupar com a gestão dos clientes.

Aplicativo Android

O aplicativo de controle foi desenvolvido em Kotlin no Android Studio. A interface do usuário foi projetada para ser intuitiva, contando com um joystick virtual para o controle de movimentação do veículo e botões para o controle da câmera e de um LED auxiliar. A comunicação com o hardware é indireta: o aplicativo escreve os comandos de controle em nós específicos no Firebase Realtime Database (/motor, /servo, /led). Para a visualização do vídeo, o app utiliza um componente WebView que se conecta ao servidor Flask, recebendo o fluxo de imagens em tempo real transmitido pelo Socket.IO. A autenticação de usuários é gerenciada pelo Firebase Authentication, garantindo que apenas usuários autorizados possam controlar o veículo.



5. RESULTADOS E DISCUSSÕES

Controle do Veículo e Câmera

O controle remoto do veículo através do joystick virtual mostrou-se responsivo e intuitivo. A velocidade do carrinho é ajustada em três níveis distintos, proporcionais à intensidade com que o usuário move o joystick, permitindo um controle de locomoção preciso. Os comandos para os servomotores da câmera e para o LED foram executados com sucesso e baixa latência.

Comunicação via Firebase

A utilização do Firebase Realtime Database como intermediário para os comandos provou ser altamente eficaz. Os tempos de resposta entre a ação no aplicativo e a execução no hardware foram mínimos, compatíveis com uma aplicação de controle em tempo real.

Transmissão de Vídeo

O sistema de streaming de vídeo funcionou conforme o esperado. O aplicativo exibiu um fluxo de imagens contínuo, permitindo o monitoramento remoto do ambiente ao redor do veículo. A qualidade da imagem foi ajustada no firmware do ESP32-CAM (`jpeg_quality = 12`) para otimizar a fluidez da transmissão em detrimento de uma resolução máxima.

Latência de Vídeo e Arquitetura

A escolha de uma arquitetura com um servidor intermediário para o vídeo foi uma decisão deliberada de design. Embora introduza uma latência ligeiramente maior em comparação com um streaming direto (dependente da qualidade da rede do ESP32, do servidor e do cliente), ela oferece a vantagem crucial da escalabilidade. O servidor pode retransmitir o feed para múltiplos clientes simultaneamente sem sobrecarregar o microcontrolador, algo que seria inviável em uma conexão direta. Esta abordagem se mostrou fundamental para a robustez da solução.

Desafios de Implementação e Soluções

Um desafio inicial foi otimizar o envio das imagens. Tentativas de enviar os dados da imagem diretamente para o Firebase se mostraram inviáveis devido às limitações de tamanho dos dados do serviço. A mudança para a arquitetura com um servidor Python/Flask via HTTP POST foi a solução que



permitiu a transmissão de um fluxo de vídeo confiável. Além disso, o ajuste fino da qualidade do JPEG no ESP32 foi essencial para balancear a qualidade visual e a velocidade de transmissão.

Usabilidade da Interface

O joystick virtual com múltiplos níveis de velocidade oferece um controle mais granular do que simples botões de "frente/trás" e "direita/esquerda", melhorando significativamente a experiência do usuário e a dirigibilidade do veículo. Há também a câmera sendo mostrada para podermos visualizar virtualmente o trajeto traçado pelo carrinho e podermos conduzir ele também com as opções de movimento "direita/esquerda" e "pra cima/pra baixo". Junto um led que tem a opção "ligar/desligar" para que em ambientes escuros que a câmera tem uma maior dificuldade de capturar o seu arredor ela tenha luz que está apontando exatamente a onde a câmera aponta. Além de ter uma seção para fazer o login do aplicativo, com uma segurança melhor para que nem todo mundo consiga controlar o spy, também mostrando que já está preparado para receber novos spy cars.

6. CONSIDERAÇÕES FINAIS

O projeto resultou em um sistema de monitoramento e controle remoto robusto e de baixo custo, que utiliza um ESP32-CAM integrado ao Firebase Realtime Database, ideal para aplicações de robótica e vigilância amadora. A comunicação em tempo real entre um aplicativo Android e o módulo é realizada através do Firebase, garantindo o controle preciso e de baixa latência dos servomotores da câmera e dos motores do veículo. A operação é facilitada por uma interface intuitiva no app, que conta com botões e um joystick virtual. Para a visualização, o streaming de vídeo é transmitido por um servidor HTTP interno do ESP32-CAM diretamente para um WebView no aplicativo. Durante o desenvolvimento, desafios importantes foram superados: para garantir um streaming mais fluido, a qualidade da imagem foi intencionalmente reduzida para aumentar a velocidade de transmissão, e a instabilidade nos comandos foi corrigida com técnicas de *debounce*. A arquitetura modular do projeto também estabelece uma base sólida que permite futuras expansões, como reconhecimento de objetos, controle por voz ou mapeamento automatizado e principalmente outro spy car. Dessa



forma, o sistema cumpriu com sucesso seus objetivos, consolidando-se como uma referência valiosa para projetos educacionais e de prototipagem rápida.

7. REFERÊNCIAS

ELETROGATE. Streaming com ESP32CAM, AsyncWebServer, PAN-TILT e Atualização OTA. Blog Eletrogate, 30 ago. 2024. Disponível em: <https://blog.eletrogate.com/streaming-com-esp32cam-asyncwebserver-pan-tilt-e-atualizacao-ota/>. Acesso em: 5 jun. 2025.

ELETROGATE. ESP32-CAM Acessado Remotamente com Ngrok. Blog Eletrogate, 13 out. 2022. Disponível em: <https://blog.eletrogate.com/esp32-cam-acessado-remotamente-com-ngrok/>. Acesso em: 6 jun. 2025.

GOOGLE. Firebase Realtime Database Documentation. 2024. Disponível em: <https://firebase.google.com/docs/database>. Acesso em: 15 jun. 2025.

VAZ, Diego Henrique. ESP32-CAM Projects and Applications. 2. ed. São Paulo: Novatec, 2023. ISBN 978-65-5580-123-8.

ESP32-CAM Save Picture in Firebase Storage Disponível em: <https://randomnerdtutorials.com/esp32-cam-save-picture-firebase-storage/>. Acesso em: 30 jun. 2025.

ESP32-CAM Video Streaming Web Server (works with Home Assistant) Disponível em: <https://randomnerdtutorials.com/esp32-cam-video-streaming-web-server-camera-home-assistant/>. Acesso em: 12 jun. 2025.

ESP32-CAM Video Streaming and Face Recognition with Arduino IDE Disponível em: <https://randomnerdtutorials.com/esp32-cam-video-streaming-face-recognition-arduino-ide/#more-82418>. Acesso em: 8 jun. 2025